

Sommaire

Sommaire.....	1
Chapitre 1 : Définir et manipuler des données types en mémoire.....	5
Déclaration de variables.....	5
Types de données scalaires - int, float, bool, str.....	5
Types de données agrégés - list, tuple, dict, set.....	6
Valeurs littérales.....	6
Portée de variables - globale, locale.....	6
Exemple de synthèse.....	7
Exercices.....	7
Chapitre 2 : Les opérateurs.....	8
Les opérateurs d'affectation.....	8
Opérateurs arithmétiques.....	8
Opérateurs relationnels.....	8
Opérateurs logiques.....	9
Exemple de synthèse.....	9
Exercices.....	10
Chapitre 3 : Convertir les données - casting ou transtypage.....	11
Casting et conversion.....	11
Conversions implicites vs explicites.....	11
Exemple de synthèse.....	11
Exercices.....	12
Chapitre 4 : Manipulation des chaines de caractères.....	13
Gestion des chaines.....	13
Formatage.....	13
Slicing.....	13

Expressions régulières.....	14
Exemple de synthèse.....	14
Exercices.....	15
Chapitre 5 : Manipulation des données structurées - list, dict et set.....	16
Gestion des listes.....	16
Gestion des dict.....	16
Gestion des set.....	16
Exemple de synthèse.....	17
Exercices.....	17
Chapitre 6 : Les instructions contrôles.....	18
Structure conditionnelles if/else/then,match.....	18
Structure match.....	18
Boucles - for/else.....	19
Boucles - while.....	19
Boucles - avec range.....	19
Boucles - avec zip.....	20
Boucles - avec items() pour les dictionnaires.....	20
Exemple de synthèse.....	20
Exercices.....	21
Chapitre 7 : Les fonctions et passage d'arguments.....	22
Les fonctions, arguments , passage par valeur, return.....	22
Arguments et valeurs par défaut.....	22
Arguments en tant que tuple.....	23
Arguments avec *args.....	23
Arguments *kargs.....	23
Fonction en tant qu'argument (delegate).....	23
Exemple de synthèse.....	24

Exercices.....	24
Chapitre 8 : Traiter une masse de données.....	25
Fonction anonyme Lambda.....	25
Traitement avec filter.....	25
Traitement avec map.....	26
Traitement avec reduce.....	26
Exemple de synthèse.....	26
Exercices.....	27
Chapitre 9 : Les générateurs.....	28
Boucle et instruction yield.....	28
Générateur prédéfinis.....	28
Exemple de synthèse.....	29
Exercices.....	29
Chapitre 10 : Un code plus robuste en prenant en compte les erreurs.....	30
Gestion des exceptions : try catch finally et throw.....	30
Instruction finally.....	30
Emettre une exception avec throw.....	31
Exemple de synthèse.....	31
Exercices.....	32
Chapitre 11 : Script python en ligne de commande et passage d'arguments.....	33
Scripts et main.....	33
Scripts et passage d'arguments.....	33
Gestion de package - pip.....	34
Exemple de synthèse.....	34
Exercices.....	35
Chapitre 12 : Gestion des package - import et création.....	36
librairie et script pip.....	36

importation de package.....	36
Contenu d'un package.....	37
chemin d'accès.....	37
package standards os, os.path, path et zlib.....	37
Gestion des répertoires - mkdir, listdir, walk, move, rmdir.....	37
Gestion des fichiers - open, read, write, seek, tell, zip.....	38
Automatiser une installation avec gel et requirements.txt.....	38
Exemple de synthèse.....	38
Exercices.....	39
Chapitre 13 : Environnement virtuel adapté à chaque application.....	40
Création d'un context python isolé avec venv.....	40
Script activate/deactivate.....	40
Exemple de synthèse.....	40
Exercices.....	41
Chapitre 14 : Gestion des fichiers et des répertoires.....	42
Concepts généraux sur les streams et fichiers.....	42
Créer un fichier text en unicode - ouvrir, ecrire, lire.....	42
Exemple de synthèse.....	42
Exercices.....	43
Chapitre 15 : Programmation orientée objets.....	44
Approche de l'orienté objets.....	44
Composition d'une classe - constructeur, méthodes et données.....	44
Objet et instance de class - self, super.....	45
Héritage de classes et redéfinition.....	45
Packages et imports et classes.....	45
Exemple de synthèse.....	46
Exercices.....	46

Chapitre 16 : Accès aux bases de données.....	47
Concepts de base.....	47
Connexion et paramétrage - connexion, cursor.....	47
Gestion de la Structure de données - Table DDL.....	48
Gestion des transactions - commit.....	48
Manipulation des données - Select/Where DML.....	48
Exemple de synthèse.....	49
Exercices.....	50

Chapitre 1 : Définir et manipuler des données types en mémoire

Maîtriser les types de données et leur manipulation en mémoire est essentiel pour stocker et traiter efficacement l'information. Ces concepts fondamentaux garantissent la rigueur et la logique de vos programmes en Python. Dans ce chapitre :

- Déclaration et affectation de variables
 - Types scalaires (int, float, bool, str)
 - Types agrégés (list, tuple, dict, set)
 - Valeurs littérales et portée des variables (locale, globale)
-

Déclaration de variables

En Python, la déclaration d'une variable se fait simplement par affectation d'une valeur à un nom, sans nécessiter de typage explicite préalable. Le type est déduit dynamiquement par l'interpréteur lors de l'exécution.

```
# Déclaration et initialisation de variables de types différents
age = 42           # Un entier (int)
nom = "Alice"      # Une chaîne de caractères (str)
```

Le nom d'une variable doit commencer par une lettre ou un tiret bas (`_`) et ne peut pas utiliser un mot-clé réservé du langage (comme `if`, `def`, `class`).

Types de données scalaires - int, float, bool, str

Les types scalaires représentent des valeurs uniques et atomiques, non décomposables en sous-éléments. Ils constituent la base de toute manipulation numérique, textuelle ou logique.

```
temperature = 23.5      # Flottant (float) pour les décimaux
est_valide = True        # Booléen (bool) valant True ou False
```

Utilisez toujours des noms explicites pour vos variables scalaires afin d'améliorer la lisibilité immédiate du code par l'équipe projet.

Types de données agrégés - list, tuple, dict, set

Les types agrégés permettent de regrouper plusieurs valeurs au sein d'une seule structure de données en mémoire. Leur choix dépend de la nécessité d'ordre, de modification ou d'unicité des éléments.

```
utilisateurs = ["Alice", "Bob", "Charlie"] # Liste modifiable (list)
coordonnees = (10.0, 20.0)                 # Tuple immuable (tuple)
```

Privilégiez les tuples pour des données fixes qui ne doivent pas être altérées au cours de l'exécution du programme, garantissant ainsi l'intégrité des structures.

Valeurs littérales

Une valeur littérale correspond à la représentation directe d'une donnée constante inscrite textuellement dans le code source du programme. Elle permet d'assigner des valeurs figées sans calcul préalable.

```
seuil_maximal = 100      # 100 est une valeur littérale entière
message_erreur = "Erreur 404" # La chaîne est une valeur littérale textuelle
```

Évitez les “nombres magiques” en remplaçant les valeurs littérales numériques répétées par des constantes explicites en début de script.

Portée de variables - globale, locale

La portée d'une variable détermine la zone du code où cette variable est accessible en lecture et en écriture. Une variable locale n'existe que dans la fonction où elle est définie, tandis qu'une variable globale est accessible dans tout le module.

```
TAXE_GLOBALE = 0.20 # Variable globale

def calculer_total(prix_ht):
    tva = prix_ht * TAXE_GLOBALE # 'tva' est locale à la fonction
    return prix_ht + tva
```

Limitez au maximum l'utilisation de variables globales pour éviter les effets de bord imprévisibles et faciliter la maintenance du code.

Exemple de synthèse

```
# Programme complet combinant variables, types scalaires, agrégés et portée
TAUX_REDUCTION = 0.15 # Variable globale

def traiter_commande(client, articles_prix):
    """Calcule le montant total d'une commande avec application d'une réduction."""
    total_brut = sum(articles_prix) # Utilisation d'un type agrégé (list)
    est_fidele = True # Type scalaire booléen

    if est_fidele:
        montant_final = total_brut * (1 - TAUX_REDUCTION) # Variable locale
    else:
        montant_final = total_brut

    return f"Client : {client} | Total à payer : {montant_final}€"

# Appel de la fonction avec des valeurs littérales
print(traiter_commande("Karim", [45.0, 15.5, 30.0]))
```

Exercices

1. **Exercice 1 :** Écrivez un script qui déclare quatre variables de types scalaires différents (int, float, bool, str), puis affichez le type de chacune d'elles à l'aide de la fonction `type()`.
2. **Exercice 2 :** Créez une liste contenant les notes d'un étudiant, écrivez une fonction qui calcule la moyenne de ces notes, et stockez le résultat dans une variable locale avant de le retourner.

Chapitre 2 : Les opérateurs

Comprendre et utiliser les opérateurs permet d'effectuer des calculs, de comparer des valeurs et de combiner des conditions logiques en Python. Ces mécanismes fondamentaux constituent les briques de base de toute logique algorithmique. Dans ce chapitre :

- Opérateurs d'affectation
 - Opérateurs arithmétiques
 - Opérateurs relationnels
 - Opérateurs logiques
-

Les opérateurs d'affectation

Les opérateurs d'affectation permettent d'attribuer une valeur à une variable en mémoire, parfois en combinant cette affectation avec une opération mathématique. Ils simplifient l'écriture des mises à jour de variables.

```
# Affectation simple et affectation augmentée
score = 10      # Affectation simple de la valeur 10
score += 5      # Équivalent à score = score + 5
```

L'opérateur d'affectation s'évalue de droite à gauche : la valeur ou le résultat de l'expression à droite est stocké dans la variable située à gauche.

Opérateurs arithmétiques

Les opérateurs arithmétiques réalisent les calculs mathématiques usuels sur des types numériques (entiers et flottants). Ils permettent de manipuler des données quantitatives au sein des programmes.

```
# Opérations arithmétiques de base
somme = 15 + 5      # Addition (vaut 20)
division = 10 / 4    # Division flottante (vaut 2.5)
```

Utilisez l'opérateur modulo (%) pour obtenir le reste d'une division entière, ce qui est particulièrement utile pour tester la parité d'un nombre.

Opérateurs relationnels

Les opérateurs relationnels comparent deux valeurs ou expressions et retournent systématiquement un résultat booléen (True ou False). Ils sont indispensables pour orienter l'exécution du code selon les conditions.

Comparaisons de valeurs

```
age = 18
majeur = age >= 18    # Retourne True car 18 est supérieur ou égal à 18
```

Veillez à ne pas confondre l'opérateur d'égalité (==) avec l'opérateur d'affectation (=) pour éviter des erreurs de logique.

Opérateurs logiques

Les opérateurs logiques permettent de combiner plusieurs expressions booléennes pour former des conditions complexes. Ils évaluent les relations à l'aide des opérateurs fondamentaux and, or et not.

Combinaison de conditions logiques

```
a_l_ecole = True
a_ses_affaires = True
peut_partir = a_l_ecole and a_ses_affaires # Vaut True si les deux conditions sont réunies
```

Python utilise l'évaluation paresseuse (*short-circuit*) pour les opérateurs logiques : l'évaluation s'arrête dès que le résultat final est déterminé.

Exemple de synthèse

Programme complet combinant affectation, arithmétique, relations et logique

```
stock_initial = 50 # Opérateur d'affectation
ventes = 12 # Valeur littérale
```

Opérateur arithmétique de soustraction combiné à l'affectation

```
stock_initial -= ventes # stock_initial vaut maintenant 38
```

```
seuil_critique = 10
rupture_imminente = False
```

Opérateurs relationnels et logiques

```
alerte_stock = (stock_initial <= seuil_critique) or rupture_imminente
```

```
print(f'stock restant : {stock_initial} | Alerte active : {alerte_stock})
```

Exercices

1. **Exercice 1 :** Écrivez un script qui initialise deux variables numériques, puis utilisez les opérateurs arithmétiques pour calculer leur somme, leur produit et le reste de leur division entière.
2. **Exercice 2 :** Déclarez une variable représentant l'âge d'un utilisateur et une autre indiquant s'il possède une autorisation. Utilisez des opérateurs relationnels et logiques pour vérifier s'il remplit les conditions d'accès (âge supérieur ou égal à 18 et autorisation vraie).

Chapitre 3 : Convertir les données - casting ou transtypage

La conversion de données, ou casting, permet de transformer un type de données en un autre pour assurer la compatibilité entre variables. Maîtriser ces conversions est indispensable pour traiter des entrées utilisateur ou fusionner des informations de natures différentes. Dans ce chapitre :

- Casting et conversion de types
 - Conversions implicites versus explicites
-

Casting et conversion

Le casting consiste à transformer explicitement une valeur d'un type donné vers un autre type (par exemple d'une chaîne de caractères vers un entier). Cette opération est indispensable pour manipuler des données textuelles provenant d'interfaces ou de fichiers.

```
# Conversion explicite d'une chaîne en entier
saisie_utilisateur = "25"
age = int(saisie_utilisateur)
```

Assurez-vous que le contenu de la chaîne est syntaxiquement convertible avant d'appliquer une fonction de casting, sous peine de déclencher une exception de type `ValueError`.

Conversions implicites vs explicites

Python réalise parfois des conversions implicites automatiques sans intervention du développeur pour éviter les pertes de données, tandis que les conversions explicites exigent l'appel direct à des fonctions dédiées comme `int()`, `float()` ou `str()`.

```
# Conversion implicite d'un entier en flottant lors d'une opération mixte
resultat = 3 + 4.5 # L'entier 3 est converti implicitement en 3.0
(résultat : 7.5)
```

Privilégiez toujours les conversions explicites pour rendre votre code prévisible et éviter les ambiguïtés de calcul entre types numériques.

Exemple de synthèse

```
# Programme complet illustrant le casting et la conversion de données
prix_article_str = "49.99" # Donnée brute sous forme de texte
quantite_str = "3" # Donnée brute sous forme de texte
```

```
# Conversions explicites pour permettre les calculs arithmétiques
prix_unit = float(prix_article_str)
quantite = int(quantite_str)

# Conversion implicite lors du calcul du sous-total
sous_total = prix_unit * quantite # float * int donne un float

# Conversion explicite inverse pour concaténation textuelle dans le message final
message = "Montant total à régler : " + str(sous_total) + " €"
print(message)
```

Exercices

1. **Exercice 1 :** Écrivez un script qui prend une chaîne de caractères représentant un prix avec des décimales, la convertit en type `float`, lui applique une taxe de 20%, puis convertit le résultat final en `str` pour l'afficher avec un message explicite.
2. **Exercice 2 :** Déclarez une variable entière et une variable flottante, effectuez une addition entre les deux, puis vérifiez et affichez le type de la variable résultante pour observer la conversion implicite de Python.

Chapitre 4 : Manipulation des chaînes de caractères

La manipulation des chaînes de caractères permet de traiter, formater et analyser des données textuelles en Python. Maîtriser ces outils est indispensable pour interagir avec les utilisateurs et structurer des messages lisibles. Dans ce chapitre :

- Gestion des chaînes
 - Formatage
 - Slicing
 - Expressions régulières
-

Gestion des chaînes

La gestion des chaînes de caractères repose sur l'utilisation de guillemets simples ou doubles pour déclarer du texte en mémoire. Python fournit de nombreuses méthodes intégrées pour transformer, nettoyer ou rechercher des motifs textuels.

```
# Manipulation de base d'une chaîne de caractères
texte = "  formation python  "
texte_propre = texte.strip().capitalize()
```

Les chaînes de caractères en Python sont immuables : toute modification textuelle génère un nouvel objet en mémoire plutôt que de modifier la chaîne originale.

Formatage

Le formatage permet d'insérer dynamiquement des variables ou des expressions au sein d'une chaîne de caractères de manière lisible et performante. Les f-strings constituent la méthode moderne recommandée en Python.

```
# Utilisation des f-strings pour l'interpolation de variables
langage = "Python"
version = 3.10
message = f"Apprentissage de {langage} en version {version}"
```

Préférez toujours l'utilisation des f-strings par rapport aux anciennes méthodes de formatage (% ou .format()) pour gagner en lisibilité et en performance.

Slicing

Le slicing (ou découpage) permet d'extraire une portion spécifique d'une chaîne de caractères en spécifiant des indices de début, de fin et de pas.

```
# Extraction d'une sous-chaîne par découpage
code_complet = "PYTH-2026"
prefixe = code_complet[:4] # Extrait "PYTH"
```

En Python, les indices de découpage commencent à zéro et l'indice de fin spécifié est toujours exclus du résultat extrait.

Expressions régulières

Les expressions régulières permettent de rechercher, valider ou extraire des motifs complexes dans des chaînes de caractères en s'appuyant sur le module standard `re`.

```
import re

# Validation d'un format de code postal à 5 chiffres
code_postal = "75001"
est_valide = bool(re.match(r"^\d{5}$", code_postal))
```

Testez toujours vos expressions régulières sur des cas limites avant de les intégrer en production pour éviter des comportements inattendus sur les données textuelles.

Exemple de synthèse

```
import re

# Programme complet combinant gestion, formatage, slicing et expressions régulières
reference_brute = "    REF-9876-FR    "

# 1. Gestion : nettoyage des espaces superflus et mise en majuscules
reference_nette = reference_brute.strip()

# 2. Slicing : extraction de la portion numérique centrale
code_numerique = reference_nette[4:8]

# 3. Expressions régulières : validation du format global
pattern = r"^REF-\d{4}-[A-Z]{2}$"
est_conforme = bool(re.match(pattern, reference_nette))

# 4. Formatage : construction du message final avec une f-string
rapport = f"Référence : {reference_nette} | Code extrait : {code_numerique} | Conforme : {est_conforme}"
print(rapport)
```

Exercices

1. **Exercice 1 :** Écrivez un script qui prend une chaîne de caractères contenant des espaces superflus et du texte en minuscules, puis utilisez les méthodes de gestion pour la nettoyer et la mettre entièrement en majuscules.
2. **Exercice 2 :** Déclarez une chaîne contenant un numéro de téléphone sous la forme d'une phrase, puis utilisez le slicing pour extraire les deux premiers caractères et formotez un message personnalisé à l'aide d'une f-string.

Chapitre 5 : Manipulation des données structurées - list, dict et set

La manipulation des données structurées permet d'organiser, de stocker et de parcourir efficacement des collections d'éléments en Python. Maîtriser ces structures est indispensable pour traiter des volumes d'informations complexes. Dans ce chapitre :

- Gestion des listes
 - Gestion des dictionnaires (dict)
 - Gestion des ensembles (set)
-

Gestion des listes

Les listes permettent de stocker une collection ordonnée d'éléments modifiables. Elles autorisent les doublons et offrent de nombreuses méthodes pour ajouter, supprimer ou trier des données en mémoire.

```
# Création et modification d'une liste
fruits = ["pomme", "banane"]
fruits.append("orange") # Ajout d'un élément à la fin
```

Utilisez la compréhension de liste pour filtrer ou transformer rapidement les éléments d'une liste de manière élégante et performante.

Gestion des dict

Les dictionnaires stockent des données sous forme de paires clé-valeur, permettant un accès ultra-rapide aux valeurs grâce à leurs clés uniques. Ils sont parfaits pour représenter des objets ou des configurations.

```
# Déclaration et accès dans un dictionnaire
utilisateur = {"nom": "Alice", "age": 30}
ville = utilisateur.get("ville", "Inconnue") # Évite une erreur si la clé n'existe pas
```

Privilégiez l'utilisation de la méthode `.get()` pour interroger un dictionnaire lorsque la clé recherchée est susceptible de ne pas y figurer.

Gestion des set

Les ensembles (set) stockent des collections non ordonnées d'éléments uniques. Ils suppriment automatiquement les doublons et s'avèrent extrêmement utiles pour effectuer des opérations mathématiques ensemblistes (union, intersection).

```
# Création d'un ensemble et suppression des doublons
nombres = {1, 2, 2, 3, 4} # Le doublon '2' est automatiquement éliminé
nombres.add(5)
```

Utilisez les opérateurs ensemblistes comme & pour l'intersection ou | pour l'union afin de comparer rapidement des collections de données.

Exemple de synthèse

```
# Programme complet combinant listes, dictionnaires et ensembles
# 1. Gestion des listes : stockage des identifiants de connexions successives
historique_connexions = ["user_1", "user_2", "user_1", "user_3"]

# 2. Gestion des set : extraction des utilisateurs uniques sans doublons
utilisateurs_uniques = set(historique_connexions)

# 3. Gestion des dict : association d'un statut à chaque utilisateur unique
statuts_utilisateurs = {
    "user_1": "actif",
    "user_2": "inactif",
    "user_3": "actif"
}

print(f"Utilisateurs uniques : {utilisateurs_uniques}")
print(f"Statut de user_1 : {statuts_utilisateurs.get('user_1')}")
```

Exercices

1. **Exercice 1 :** Créez une liste contenant plusieurs prénoms avec des doublons, convertissez-la en ensemble (set) pour éliminer les doublons, puis affichez le résultat.
2. **Exercice 2 :** Déclarez un dictionnaire représentant un produit (avec les clés nom, prix et stock), puis écrivez une instruction pour mettre à jour la valeur du stock.

Chapitre 6 : Les instructions contrôles

Les instructions de contrôle permettent d'orienter le flux d'exécution d'un programme en fonction de conditions et de répéter des blocs de instructions. Maîtriser ces structures est indispensable pour automatiser des tâches complexes. Dans ce chapitre :

- Structures conditionnelles `if/else` et `match`
 - Boucles itératives (`for/else`, `while`)
 - Itérations avancées (`range`, `zip`, `items()`)
-

Structure conditionnelles `if/else/then,match`

Les structures conditionnelles permettent d'exécuter des blocs de code différents selon la validité d'une ou plusieurs conditions logiques. L'instruction `match`, introduite récemment, facilite les aiguillages complexes par motif.

```
# Utilisation d'une structure conditionnelle classique
note = 14
if note >= 10:
    statut = "Admis"
else:
    statut = "Recalé"
```

Privilégiez l'utilisation de `elif` pour enchaîner plusieurs conditions mutuellement exclusives proprement sans imbriquer excessivement vos blocs `if`.

Structure `match`

L'instruction `match` réalise un filtrage par motif (*pattern matching*), permettant de comparer une valeur à plusieurs structures ou cas possibles de manière très lisible.

```
# Utilisation de match pour aiguiller selon une commande
commande = "quit"
match commande:
    case "start":
        print("Démarrage...")
    case "quit":
        print("Arrêt...")
    case _:
        print("Commande inconnue")
```

Utilisez le motif universel `_` comme dernier cas dans un `match` pour capturer toutes les valeurs non gérées explicitement.

Boucles - for/else

La boucle `for` permet de parcourir séquentiellement les éléments d'une collection. En Python, elle peut être associée à un bloc `else` optionnel qui s'exécute si la boucle s'est terminée sans interruption par un `break`.

```
# Parcours d'une liste avec une boucle for
nombres = [1, 2, 3]
for n in nombres:
    print(n)
```

Utilisez le bloc `else` d'une boucle `for` pour exécuter du code de validation si aucun élément recherché n'a déclenché de rupture anticipée.

Boucles - while

La boucle `while` répète l'exécution d'un bloc d'instructions tant qu'une condition booléenne associée reste évaluée à `True`. Elle est idéale lorsque le nombre d'itérations n'est pas connu à l'avance.

```
# Compteur avec une boucle while
compteur = 0
while compteur < 3:
    print(compteur)
    compteur += 1
```

Veillez à toujours faire évoluer la variable de condition à l'intérieur d'une boucle `while` pour éviter les boucles infinies.

Boucles - avec range

L'association d'une boucle `for` avec la fonction `range()` permet de répéter un bloc d'instructions un nombre précis de fois en générant une séquence numérique efficace.

```
# Répétition d'une action à l'aide de range
for i in range(3):
    print(f"Itération numéro {i}")
```

La fonction `range(debut, fin, pas)` accepte des arguments optionnels pour démarrer à un autre indice ou parcourir les éléments par pas spécifiques.

Boucles - avec zip

La fonction `zip()` permet de parcourir simultanément plusieurs collections en assemblant leurs éléments sous forme de tuples, ce qui simplifie le traitement croisé de données.

```
# Itération conjointe sur deux listes
noms = ["Alice", "Bob"]
scores = [85, 92]
for nom, score in zip(noms, scores):
    print(f"{nom} a obtenu {score} points")
```

Si les collections passées à `zip()` n'ont pas la même longueur, l'itération s'arrête automatiquement dès que la plus courte est épuisée.

Boucles - avec items() pour les dictionnaires

La méthode `.items()` permet de parcourir à la fois les clés et les valeurs d'un dictionnaire lors d'une même boucle `for`, optimisant la lecture des données structurées.

```
# Parcours des clés et valeurs d'un dictionnaire
parametres = {"theme": "sombre", "volume": 80}
for cle, valeur in parametres.items():
    print(f"{cle} : {valeur}")
```

Utilisez `.items()` dès que vous avez besoin de manipuler simultanément la clé et sa valeur associée pour éviter des appels d'accès superflus.

Exemple de synthèse

```
# Programme complet combinant conditions, boucles, range, zip et items
utilisateurs = ["Alice", "Bob", "Charlie"]
points = [45, 60, 30]

# 1. Boucle avec range pour un affichage numéroté
for i in range(len(utilisateurs)):
    print(f"Rang {i + 1}")

# 2. Boucle avec zip pour associer utilisateurs et scores
for user, score in zip(utilisateurs, points):
    # 3. Structure conditionnelle classique
    if score >= 50:
        niveau = "Expert"
    else:
```

```
niveau = "Débutant"
print(f"{user} ({niveau}) avec {score} pts")

# 4. Boucle avec items() pour parcourir un dictionnaire de
# configuration
config = {"mode": "admin", "debug": True}
for parametre, etat in config.items():
    match parametre:
        case "mode":
            print(f"Mode actif : {etat}")
        case "debug":
            print(f"Mode débogage activé : {etat}")
```

Exercices

1. **Exercice 1 :** Écrivez un script utilisant une boucle `for` et `range()` pour afficher uniquement les nombres pairs de 0 à 10.
2. **Exercice 2 :** Créez un dictionnaire associant des noms de fruits à leur prix, puis utilisez la méthode `.items()` dans une boucle pour afficher chaque fruit et son prix avec une structure conditionnelle vérifiant s'il est supérieur à un certain seuil.

Chapitre 7 : Les fonctions et passage d'arguments

Les fonctions permettent de modulariser le code en regroupant des instructions réutilisables sous un même nom. Maîtriser le passage d'arguments et les structures de retour est indispensable pour concevoir des programmes propres et maintenables. Dans ce chapitre :

- Définition des fonctions, arguments, passage et valeur de retour (return)
 - Arguments et valeurs par défaut
 - Arguments variables via les tuples, *args et **kwargs
 - Fonctions en tant qu'arguments (délégués)
-

Les fonctions, arguments , passage par valeur, return

Une fonction se déclare avec le mot-clé `def` suivi d'un nom et de parenthèses. Elle accepte des arguments en entrée et peut renvoyer un résultat grâce à l'instruction `return`.

Déclaration d'une fonction simple avec retour

```
def additionner(a, b):  
    resultat = a + b  
    return resultat
```

En Python, les objets sont passés par affectation : modifier un objet mutable à l'intérieur d'une fonction se répercute en dehors, contrairement aux objets immuables.

Arguments et valeurs par défaut

Les arguments par défaut permettent de définir une valeur de repli lorsqu'un paramètre n'est pas explicitement fourni lors de l'appel de la fonction.

Fonction avec un argument doté d'une valeur par défaut

```
def saluer(nom, message="Bonjour"):  
    return f"{message}, {nom} !"
```

Ne jamais utiliser d'objets mutables (comme des listes ou des dictionnaires) comme valeurs par défaut d'une fonction, car leur état serait conservé entre les appels successifs.

Arguments en tant que tuple

Il est possible de regrouper plusieurs valeurs d'arguments dans un tuple pour les manipuler de manière globale au sein de la fonction.

```
# Fonction acceptant un tuple d'éléments regroupés
def afficher_coordonnees(coord):
    x, y = coord
    return f"Position X: {x}, Y: {y}"
```

Utilisez l'emballage de tuples lorsque vos données possèdent une structure fixe et ordonnée que vous souhaitez traiter en bloc.

Arguments avec *args

La syntaxe `*args` permet de transmettre un nombre variable d'arguments positionnels non nommés à une fonction, qui les récupère automatiquement sous forme de tuple.

```
# Fonction acceptant un nombre indéfini d'arguments positionnels
def sommer_tout(*args):
    return sum(args)
```

Le nom `args` est une convention en Python, mais c'est l'astérisque `*` qui indique au langage de capturer tous les arguments positionnels excédentaires.

Arguments *kwargs

La syntaxe `**kwargs` (souvent appelée `kargs`) permet de récupérer un nombre variable d'arguments nommés sous la forme d'un dictionnaire au sein de la fonction.

```
# Fonction acceptant des arguments nommés dynamiques
def configurer(**kwargs):
    for cle, valeur in kwargs.items():
        print(f"{cle} = {valeur}")
```

L'utilisation conjointe de `*args` et `**kwargs` offre une flexibilité maximale pour créer des fonctions enveloppes (*wrappers*) ou des décorateurs.

Fonction en tant qu'argument (delegate)

En Python, les fonctions sont des objets de première classe, ce qui signifie qu'elles peuvent être passées en tant qu'arguments à d'autres fonctions, agissant ainsi comme des délégués.

Utilisation d'une fonction en tant qu'argument

```
def appliquer_operation(operation, x, y):
    return operation(x, y)
```

```
resultat = appliquer_operation(lambda a, b: a * b, 4, 5)
```

Passer des fonctions en argument est la base de la programmation fonctionnelle et permet de concevoir des algorithmes hautement génériques et réutilisables.

Exemple de synthèse

*# Programme complet combinant fonctions, valeurs par défaut, *args, **kwargs et délégués*

```
def calculer_total_remise(taux=0.1, *montants, **details):
    """Calcule un total avec *args et affiche les options via
    **kwargs."""
    sous_total = sum(montants)
    total_net = sous_total * (1 - taux)

    print(f"Facture pour {details.get('client', 'Client inconnu')}")
    print(f"Sous-total : {sous_total}€ | Net : {total_net}€")
    return total_net
```

Fonction déléguée à passer en paramètre

```
def formater_monnaie(montant):
    return f"{montant:.2f} EUR"
```

Appel de la fonction principale avec différents types d'arguments

```
montant_final = calculer_total_remise(0.2, 100.0, 50.0, 25.0,
client="Alice", mode="express")
print("Format final :", formater_monnaie(montant_final))
```

Exercices

1. **Exercice 1 :** Écrivez une fonction qui accepte un nombre indéfini d'entiers via `*args` et retourne leur moyenne arithmétique.
2. **Exercice 2 :** Créez une fonction qui prend en paramètre une fonction mathématique et deux nombres, puis applique cette fonction sur les deux nombres pour retourner le résultat.

Chapitre 8 : Traiter une masse de données

Le traitement de masses de données permet d'appliquer des transformations et des filtres performants sur des collections en Python. Maîtriser ces outils fonctionnels est indispensable pour manipuler efficacement des flux d'informations importants. Dans ce chapitre :

- Fonctions anonymes `lambda`
 - Filtrage de données avec `filter`
 - Transformation de données avec `map`
 - Agrégation de données avec `reduce`
-

Fonction anonyme Lambda

Une fonction anonyme, introduite par le mot-clé `lambda`, permet de définir rapidement une fonction compacte sans nom sur une seule ligne de code. Elle est idéale pour des traitements courts et ponctuels.

```
# Déclaration et appel d'une fonction lambda pour calculer le carré
carre = lambda x: x ** 2
resultat = carre(5)
```

Utilisez les fonctions `lambda` principalement comme arguments pour des fonctions de traitement de collections comme `map` ou `filter`.

Traitement avec filter

La fonction `filter()` permet de filtrer les éléments d'une collection en évaluant chaque élément à l'aide d'une fonction conditionnelle qui retourne un booléen.

```
# Filtrage des nombres pairs d'une liste
nombres = [1, 2, 3, 4, 5, 6]
pairs = list(filter(lambda x: x % 2 == 0, nombres))
```

Le résultat retourné par `filter()` en Python est un itérateur, pensez à le convertir explicitement en `list` ou `tuple` pour exploiter les données.

Traitement avec map

La fonction `map()` applique une fonction spécifique à l'ensemble des éléments d'une collection itérable, transformant ainsi les données en une seule passe.

```
# Application d'une transformation pour multiplier par 2 chaque élément
valeurs = [1, 2, 3, 4]
doubles = list(map(lambda x: x * 2, valeurs))
```

Les compréhensions de listes constituent souvent une alternative plus lisible et idiomatique aux fonctions `map()` en Python.

Traitement avec reduce

La fonction `reduce()`, issue du module `functools`, permet de réduire une collection de données à une valeur unique en appliquant cumulativement une fonction binaire de manière séquentielle.

```
from functools import reduce

# Calcul de la somme des éléments d'une liste par réduction
nombres = [1, 2, 3, 4]
somme_totale = reduce(lambda x, y: x + y, nombres)
```

Pensez à importer `reduce` depuis le module `functools` avant de l'utiliser, car cette fonction n'est plus intégrée directement dans l'espace de noms global de Python.

Exemple de synthèse

```
from functools import reduce

# Programme complet combinant lambda, filter, map et reduce sur une masse de données
prix_articles = [12.5, 45.0, 8.0, 100.0, 32.5]

# 1. Filtrer les articles dont le prix est supérieur à 15.0 via filter et lambda
articles_cibles = list(filter(lambda p: p > 15.0, prix_articles))

# 2. Appliquer une remise de 10% sur ces articles via map et lambda
prix_remises = list(map(lambda p: p * 0.9, articles_cibles))

# 3. Calculer le montant total cumulé de ces articles via reduce et lambda
```

```
montant_global = reduce(lambda total, p: total + p, prix_remises, 0.0)

print(f"Articles remisés : {prix_remises}")
print(f"Montant global de la commande : {montant_global:.2f} €")
```

Exercices

1. **Exercice 1 :** Utilisez la fonction `filter()` associée à une expression `lambda` pour extraire uniquement les mots de longueur supérieure à 5 caractères d'une liste de chaînes.
2. **Exercice 2 :** Importez `reduce` depuis `functools` et écrivez un script qui calcule le produit de tous les éléments d'une liste d'entiers.

Chapitre 9 : Les générateurs

Les générateurs permettent de produire des séquences de valeurs à la demande sans stocker l'intégralité des données en mémoire. Maîtriser ces concepts est indispensable pour optimiser l'efficacité de vos programmes lors du traitement de grands volumes d'informations. Dans ce chapitre :

- Boucle et instruction `yield`
 - Générateurs prédéfinis
-

Boucle et instruction `yield`

L'instruction `yield` permet à une fonction de retourner une valeur tout en suspendant son état d'exécution, transformant ainsi la fonction en un générateur capable de reprendre là où il s'était arrêté.

Définition d'une fonction génératrice simple

```
def generer_nombres(limite):  
    n = 0  
    while n < limite:  
        yield n  
        n += 1
```

Contrairement à `return`, l'instruction `yield` conserve l'état local de la fonction entre chaque itération, ce qui économise considérablement la mémoire vive.

Générateur prédéfinis

Les générateurs prédéfinis englobent les expressions génératrices et les structures intégrées de Python qui produisent des flux d'éléments de manière paresseuse, évitant l'allocation préalable d'une collection complète.

Utilisation d'une expression génératrice pour un calcul optimisé en mémoire

```
carres_gen = (x ** 2 for x in range(5))  
premier_element = next(carres_gen)
```

Privilégiez les expressions génératrices entre parenthèses plutôt que les compréhensions de listes dès que vous manipulez des flux volumineux dont vous n'avez pas besoin de stocker l'ensemble des résultats simultanément.

Exemple de synthèse

Programme complet combinant la création d'un générateur avec yield et un générateur prédéfini

```
def lecteur_lignes_simule(donnees):  
    """Générateur personnalisé pour traiter des lignes de texte à la  
    demande."""  
    for ligne in donnees:  
        # Instruction yield pour suspendre et renvoyer la ligne  
        nettoyée  
        yield ligne.strip().upper()  
  
flux_brut = ["  premiere ligne  ", "  seconde ligne  ", "  troisieme  
ligne  "]  
  
# Utilisation du générateur personnalisé  
gen_personnalise = lecteur_lignes_simule(flux_brut)  
  
# Utilisation d'un générateur prédéfini (enumerate) associé  
for index, texte_traite in enumerate(gen_personnalise, start=1):  
    print(f"Ligne {index} : {texte_traite}")
```

Exercices

1. **Exercice 1 :** Écrivez une fonction génératrice utilisant yield pour produire les nombres pairs jusqu'à une limite passée en paramètre, puis parcourez ce générateur avec une boucle for.
2. **Exercice 2 :** Créez une expression génératrice qui calcule les carrés des nombres de 1 à 10, et récupérez les valeurs un par un à l'aide de la fonction next ().

Chapitre 10 : Un code plus robuste en prenant en compte les erreurs

La gestion des erreurs permet d'anticiper et de traiter les incidents d'exécution pour empêcher l'arrêt brutal d'un programme en Python. Maîtriser ces mécanismes est indispensable pour concevoir des applications fiables et résilientes. Dans ce chapitre :

- Gestion des exceptions (try, except, finally) et levée d'exceptions (raise)
 - Utilisation de l'instruction finally
 - Émission personnalisée d'une exception
-

Gestion des exceptions : try catch finally et throw

La gestion des exceptions repose sur le bloc try pour surveiller le code à risque et except pour intercepter les erreurs survenues. En Python, le mot-clé raise équivaut au throw des autres langages pour émettre une exception.

Interception d'une division par zéro

try:

```
    resultat = 10 / 0
```

except ZeroDivisionError:

```
    resultat = "Erreur : Division par zéro impossible"
```

Spécifiez toujours le type précis d'exception à intercepter dans votre bloc except plutôt que d'utiliser une clause globale muette qui masquerait des bugs inattendus.

Instruction finally

L'instruction finally permet de définir un bloc de code qui s'exécute systématiquement à la fin, qu'une exception ait été levée ou non. Elle est idéale pour libérer des ressources (fichiers, connexions réseau).

Utilisation de finally pour la clôture des ressources

try:

```
    fichier = ouvrir_fichier("donnees.txt")
```

except FileNotFoundError:

```
    print("Fichier introuvable")
```

finally:

```
    fermer_fichier() # Exécuté dans tous les cas
```

Privilégiez l'utilisation du gestionnaire de contexte `with` lorsque c'est possible pour automatiser le nettoyage des ressources sans recourir explicitement à un bloc `finally`.

Emettre une exception avec `throw`

Il est possible d'émettre volontairement une exception à l'aide de l'instruction `raise` (similaire à `throw`) pour signaler qu'une règle métier ou une condition critique n'est pas respectée.

Émission d'une exception si l'âge est invalide

```
age = -5
```

```
if age < 0:
```

```
    raise ValueError("L'âge ne peut pas être négatif")
```

Créez vos propres classes d'exceptions personnalisées en héritant de la classe `Exception` de base pour affiner la gestion des erreurs spécifiques à votre domaine métier.

Exemple de synthèse

Programme complet combinant try, except, finally et la levée d'une exception avec raise

```
def convertir_et_diviser(valeur_str, diviseur_str):
```

```
    """Convertit deux chaînes en entiers et réalise une division
    sécurisée."""
```

```
    try:
```

```
        valeur = int(valeur_str)
```

```
        diviseur = int(diviseur_str)
```

Émission d'une exception personnalisée si le diviseur est nul

```
        if diviseur == 0:
```

```
            raise ZeroDivisionError("Le diviseur ne peut pas être égal
à zéro.")
```

```
        quotient = valeur / diviseur
```

```
    except ValueError as e:
```

```
        return f"Erreur de format numérique : {e}"
```

```
    except ZeroDivisionError as e:
```

```
        return f"Erreur mathématique : {e}"
```

```
    finally:
```

```
        print("Fin de l'opération de calcul sécurisée.")
```

```
return f"Résultat du calcul : {quotient}"
```

```
# Test de la fonction avec des valeurs littérales
```

```
print(convertir_et_diviser("100", "4"))
```

```
print(convertir_et_diviser("50", "0"))
```

Exercices

1. **Exercice 1 :** Écrivez un script qui demande à l'utilisateur de saisir un nombre, utilise un bloc `try...except` pour intercepter une éventuelle erreur de saisie (`ValueError`), et affiche un message adapté.
2. **Exercice 2 :** Créez une fonction qui vérifie si un mot de passe possède au moins 8 caractères. Si ce n'est pas le cas, utilisez `raise` pour émettre une exception personnalisée de type `ValueError`.

Chapitre 11 : Script python en ligne de commande et passage d'arguments

La création de scripts en ligne de commande et le passage d'arguments permettent d'automatiser des tâches et d'interagir directement avec vos programmes Python depuis le terminal. Maîtriser ces outils est indispensable pour industrialiser vos développements. Dans ce chapitre :

- Scripts et `__main__`
 - Scripts et passage d'arguments
 - Gestion de package avec `pip`
-

Scripts et main

La structure `if __name__ == "__main__":` permet d'identifier si un fichier Python est exécuté directement comme un script principal ou importé comme un module dans un autre programme.

```
# Vérification du point d'entrée principal du script
def executer_tache():
    print("Exécution du traitement principal...")

if __name__ == "__main__":
    executer_tache()
```

Placez toujours le code d'exécution principale de vos scripts sous cette condition pour permettre la réutilisation propre de vos fonctions par d'autres modules.

Scripts et passage d'arguments

Le passage d'arguments en ligne de commande permet de transmettre des paramètres dynamiques à un script lors de son lancement depuis le terminal, notamment via le module standard `sys` ou `argparse`.

```
import sys

# Récupération des arguments passés en ligne de commande
arguments = sys.argv
nom_script = sys.argv[0]
```

Privilégiez l'utilisation du module `argparse` pour les scripts complexes afin de gérer automatiquement l'aide, les options obligatoires et les types d'arguments.

Gestion de package - pip

L'outil `pip` est le gestionnaire de paquets officiel de Python qui permet d'installer, de mettre à jour et de supprimer des bibliothèques tierces depuis le Python Package Index (PyPI).

Installation d'un package tiers en ligne de commande (exemple)
`pip install requests`

Utilisez systématiquement un environnement virtuel (`venv`) avant d'installer des packages avec `pip` pour isoler les dépendances de vos différents projets.

Exemple de synthèse

```
import sys

def traiter_commande_cli():
    """Simule un script en ligne de commande avec gestion des arguments."""
    # Vérification du point d'entrée principal
    if len(sys.argv) < 2:
        print("Erreur : Veuillez fournir un argument (ex: start ou stop).")
        return

    action = sys.argv[1].lower()

    # Traitement conditionnel basé sur l'argument reçu
    if action == "start":
        print("Démarrage du service en ligne de commande...")
    elif action == "stop":
        print("Arrêt du service demandé.")
    else:
        print(f"Action inconnue : {action}")

if __name__ == "__main__":
    traiter_commande_cli()
```

Exercices

1. **Exercice 1 :** Écrivez un script Python comportant une structure `if __name__ == "__main__":` qui affiche un message de bienvenue personnalisé lorsque le fichier est exécuté directement.
2. **Exercice 2 :** Utilisez le module `sys` pour récupérer un nom passé en argument dans le terminal et affichez une salutation personnalisée intégrant ce paramètre.

Chapitre 12 : Gestion des package - import et création

La gestion des packages et des modules permet d'organiser et de structurer le code en composants réutilisables tout en exploitant la bibliothèque standard de Python. Maîtriser ces outils est indispensable pour industrialiser vos projets. Dans ce chapitre :

- Librairie, scripts pip et importation de packages
 - Contenu d'un package et gestion des chemins d'accès
 - Packages standards (os, os.path, path et zlib)
 - Gestion des répertoires et des fichiers
 - Automatisation d'installation avec `get` et `requirements.txt`
-

librairie et script pip

Les scripts et librairies gérés via pip permettent d'installer, de maintenir et de partager des dépendances logicielles tierces au sein de vos environnements de développement.

```
# Exemple de commande d'installation via pip (à exécuter dans le terminal)  
# pip install package_name
```

Utilisez toujours des environnements virtuels isolés pour vos projets afin d'éviter les conflits de versions entre les différentes bibliothèques tierces installées.

importation de package

L'importation de modules ou de packages s'effectue à l'aide des instructions `import` ou `from ... import` pour intégrer des fonctionnalités externes dans vos scripts.

```
# Importation d'un module standard ou tiers  
import math  
racine = math.sqrt(16)
```

Évitez les importations globales du type `from module import *` pour préserver la lisibilité de votre espace de noms et éviter les conflits de noms de variables.

Contenu d'un package

Un package en Python est un répertoire contenant un fichier d'initialisation (`__init__.py`) et plusieurs modules sous forme de fichiers source, permettant de structurer une application modulaire.

```
# Structure logique d'un appel à un sous-module de package  
# from mon_package import mon_module
```

Le fichier `__init__.py` (qui peut être vide dans les versions récentes de Python) indique explicitement à l'interpréteur que le dossier doit être traité comme un package.

chemin d'accès

La gestion des chemins d'accès permet de localiser avec précision les fichiers et répertoires sur le disque dur, garantissant la portabilité de vos scripts d'un système à un autre.

```
import os
```

```
# Récupération du chemin absolu du répertoire courant  
chemin_courant = os.getcwd()
```

Privilégiez l'utilisation du module moderne `pathlib` pour manipuler les chemins d'accès de manière orientée objet et indépendante du système d'exploitation.

package standards `os`, `os.path`, `path` et `zlib`

La bibliothèque standard de Python intègre de nombreux modules puissants comme `os`, `os.path` et `zlib` pour interagir avec le système d'exploitation et compresser des données.

```
import os.path
```

```
# Vérification de l'existence d'un fichier  
existe = os.path.exists("config.json")
```

Explorez d'abord la bibliothèque standard avant d'installer des packages tiers, car elle couvre déjà la majorité des besoins basiques en manipulation système.

Gestion des répertoires - `mkdir`, `listdir`, `walk`, `move`, `rmdir`

La manipulation des répertoires permet de créer, parcourir, déplacer ou supprimer des dossiers de manière automatisée au sein de vos scripts.

```
import os
```

```
# Création d'un nouveau répertoire de travail
os.makedirs("nouveau_dossier", exist_ok=True)
```

Utilisez l'argument `exist_ok=True` lors de la création de dossiers pour éviter de déclencher une exception si le répertoire existe déjà.

Gestion des fichiers - open, read, write, seek, tell, zip

La gestion des fichiers bas niveau permet d'ouvrir, de lire, d'écrire, de positionner le curseur (seek, tell) et de manipuler des archives compressées (zip).

```
# Ouverture et écriture sécurisée dans un fichier texte
with open("journal.txt", "w", encoding="utf-8") as f:
    f.write("Premier message de journalisation.")
```

Utilisez toujours le gestionnaire de contexte `with` pour l'ouverture des fichiers afin de garantir leur fermeture automatique, même en cas d'erreur.

Automatiser une installation avec `gel` et `requirements.txt`

L'automatisation du déploiement s'appuie sur un fichier `requirements.txt` listant les dépendances exactes du projet, facilitant leur réinstallation en une seule commande.

```
# Commande pour installer toutes les dépendances d'un projet
pip install -r requirements.txt
```

Mettez régulièrement à jour votre fichier `requirements.txt` pour refléter fidèlement l'état de vos dépendances de développement.

Exemple de synthèse

```
import os
import os.path
import zlib

def preparer_environnement_projet(nom_dossier):
    """Crée un répertoire de travail, écrit un fichier de log et
    vérifie sa présence."""
    # 1. Gestion des répertoires : création sécurisée
    os.makedirs(nom_dossier, exist_ok=True)

    chemin_fichier = os.path.join(nom_dossier, "donnees.txt")
```

```
# 2. Gestion des fichiers : écriture de données
with open(chemin_fichier, "w", encoding="utf-8") as fichier:
    fichier.write("Contenu critique à compresser et archiver.")

# 3. Utilisation de packages standards (vérification et
compression zlib)
if os.path.exists(chemin_fichier):
    with open(chemin_fichier, "rb") as fichier_1:
        donnees_brutes = fichier_1.read()
        donnees_compressées = zlib.compress(donnees_brutes)
        print(f"Taille originale : {len(donnees_brutes)} octets")
        print(f"Taille compressée : {len(donnees_compressées)}
octets")

# Appel de la fonction de synthèse
preparer_environnement_projet("stock_donnees")
```

Exercices

1. **Exercice 1 :** Écrivez un script qui utilise le module `os` pour lister tous les fichiers présents dans le répertoire courant.
2. **Exercice 2 :** Créez un fichier texte, écrivez-y une phrase, puis utilisez le module `zlib` pour compresser son contenu textuel lu en mode binaire.

Chapitre 13 : Environnement virtuel adapté à chaque application

La création d'environnements virtuels permet d'isoler les dépendances de chaque application Python pour éviter les conflits entre les bibliothèques installées sur le système. Maîtriser ces outils est indispensable pour garantir la stabilité et la reproductibilité de vos projets. Dans ce chapitre :

- Création d'un contexte Python isolé avec venv
 - Utilisation des scripts d'activation et de désactivation (activate/deactivate)
-

Création d'un contexte python isolé avec venv

Le module venv permet de générer un répertoire de travail contenant une copie autonome de l'interpréteur Python et de sa bibliothèque standard. Cela isole complètement l'environnement des paquets globaux de la machine.

```
# Création d'un environnement virtuel nommé .venv dans le répertoire du projet  
python -m venv .venv
```

Nommez généralement votre environnement virtuel `.venv` pour qu'il soit facilement identifiable et ignoré par les outils de gestion de versions comme Git.

Script activate/deactivate

Les scripts `activate` et `deactivate` permettent respectivement d'activer et de quitter l'environnement virtuel pour rediriger dynamiquement l'utilisation de la commande `pip` vers le dossier isolé.

```
# Activation de l'environnement virtuel sous Linux / macOS  
source .venv/bin/activate
```

Vérifiez toujours que votre invite de commande affiche le nom de l'environnement entre parenthèses (par exemple `(.venv)`) pour confirmer son activation correcte avant d'installer vos paquets.

Exemple de synthèse

```
# Séquence complète de commandes pour configurer et utiliser un environnement virtuel sous Linux/macOS :
```

```
# 1. Création du contexte Python isolé
```

```
python -m venv mon_env
```

```
# 2. Activation de l'environnement virtuel
```

```
source mon_env/bin/activate
```

```
# 3. Installation d'une bibliothèque tierce dans cet environnement isolé
```

```
pip install requests
```

```
# 4. Sortie de l'environnement virtuel
```

```
deactivate
```

Exercices

1. **Exercice 1 :** Exécutez la commande dans votre terminal pour créer un environnement virtuel nommé `env_projet` à la racine de votre dossier de travail.
2. **Exercice 2 :** Activez l'environnement virtuel créé, vérifiez son bon fonctionnement, puis désactivez-le à l'aide de la commande appropriée.

Chapitre 14 : Gestion des fichiers et des répertoires

La gestion des fichiers et des répertoires permet d'interagir directement avec le système de stockage pour enregistrer et restituer des informations. Maîtriser ces concepts est indispensable pour persister l'état de vos applications. Dans ce chapitre :

- Concepts généraux sur les streams et fichiers
 - Créer un fichier texte en unicode : ouverture, écriture et lecture
-

Concepts généraux sur les streams et fichiers

Les streams (flux) représentent des canaux de communication séquentiels permettant de transférer des données entre la mémoire vive et des périphériques de stockage ou des fichiers. Ils garantissent un traitement fluide et ordonné des flux d'informations.

Illustration conceptuelle du traitement par flux
flux_donnees = "Lecture ou écriture séquentielle"

Considérez toujours un fichier ouvert comme un flux unidirectionnel ou bidirectionnel nécessitant une clôture rigoureuse après utilisation.

Créer un fichier text en unicode - ouvrir, écrire, lire

La création d'un fichier texte en encodage Unicode garantit la prise en charge universelle des caractères accentués et des symboles internationaux. Les fonctions natives permettent d'ouvrir, d'écrire et de lire ces contenus en toute sécurité.

```
# Création, écriture et lecture d'un fichier texte en UTF-8
with open("document.txt", "w", encoding="utf-8") as f:
    f.write("Texte en Unicode avec des accents : é, à, ê.")

with open("document.txt", "r", encoding="utf-8") as f:
    contenu = f.read()
```

Spécifiez systématiquement l'argument `encoding="utf-8"` lors de l'ouverture de fichiers texte pour éviter les erreurs de décodage selon les systèmes d'exploitation.

Exemple de synthèse

Programme complet combinant les concepts de streams et la gestion de fichiers Unicode

```
nom_fichier = "message_unicode.txt"
```

```
# 1. Écriture dans un fichier texte en Unicode
```

```
with open(nom_fichier, "w", encoding="utf-8") as fichier_sortie:
    fichier_sortie.write("Bonjour Karim !\n")
    fichier_sortie.write("Apprentissage de la gestion des flux et
fichiers en Python.\n")
```

```
# 2. Lecture du fichier texte via un flux sécurisé
```

```
with open(nom_fichier, "r", encoding="utf-8") as fichier_entree:
    for numero_ligne, ligne in enumerate(fichier_entree, start=1):
        print(f"Ligne {numero_ligne} : {ligne.strip()}")
```

Exercices

1. **Exercice 1 :** Écrivez un script qui crée un fichier texte nommé `notes.txt` en encodage Unicode, puis y inscrit une phrase comportant des caractères accentués.
2. **Exercice 2 :** Ouvrez le fichier `notes.txt` en mode lecture avec l'encodage approprié, lisez son contenu global et affichez-le dans la console.

Chapitre 15 : Programmation orientée objets

La programmation orientée objets permet de structurer un programme autour de concepts et de données modélisés sous forme de classes et d'objets. Maîtriser ces principes est indispensable pour concevoir des architectures logicielles modulaires et maintenables. Dans ce chapitre :

- Approche de l'orienté objets
 - Composition d'une classe (constructeur, méthodes et données)
 - Objets et instances de classe (`self`, `super`)
 - Héritage de classes et redéfinition
 - Packages, imports et classes
-

Approche de l'orienté objets

L'approche orientée objets consiste à regrouper au sein d'une même entité (la classe) les données (attributs) et les traitements (méthodes) qui leur sont associés. Cela favorise l'encapsulation et la réutilisabilité du code.

Modélisation conceptuelle d'un objet en Python

```
class Vehicule:  
    pass
```

Pensez vos classes comme des plans de construction permettant de donner naissance à des objets autonomes dotés de comportements spécifiques.

Composition d'une classe - constructeur, méthodes et données

Une classe se compose d'un constructeur (la méthode spéciale `__init__`), de données attributaires et de méthodes pour définir les actions que l'objet peut réaliser.

Définition d'une classe avec constructeur et méthode

```
class CompteBancaire:  
    def __init__(self, titulaire, solde):  
        self.titulaire = titulaire  
        self.solde = solde
```

Le constructeur s'exécute automatiquement lors de l'instanciation de la classe pour initialiser l'état initial des données de l'objet.

Objet et instance de class - self, super

L'instance représente un objet concret issu d'une classe. Le paramètre `self` fait référence à l'instance courante, tandis que `super()` permet d'accéder aux méthodes de la classe parente.

Utilisation de self pour lier les données à l'instance

```
class Chien:
    def __init__(self, nom):
        self.nom = nom

    def aboyer(self):
        return f"{self.nom} aboie !"
```

Utilisez systématiquement `self` comme premier paramètre de vos méthodes d'instance pour garantir l'accès correct aux attributs propres de l'objet.

Héritage de classes et redéfinition

L'héritage permet de créer une nouvelle classe (fille) à partir d'une classe existante (parente) pour réutiliser du code et redéfinir certains comportements spécifiques.

Héritage simple et spécialisation

```
class Animal:
    def emettre_son(self):
        return "Son générique"

class Chat(Animal):
    def emettre_son(self):
        return "Miaou"
```

La redéfinition de méthodes (*method overriding*) permet d'adapter le comportement d'une classe fille tout en conservant la signature de la classe parente.

Packages et imports et classes

L'organisation des classes au sein de modules et de packages permet de structurer les grands projets logiciels et de les importer proprement là où ils sont nécessaires.

Importation ciblée d'une classe depuis un module de package
from mon_package.modele import CompteBancaire

Veillez à regrouper les classes ayant des responsabilités métiers proches au sein d'un même module pour préserver la clarté de votre architecture.

Exemple de synthèse

Programme complet combinant classes, constructeur, méthodes, self, héritage et redéfinition

```
class Utilisateur:
    """Classe parente représentant un utilisateur générique."""
    def __init__(self, identifiant):
        self.identifiant = identifiant

    def obtenir_profil(self):
        return f"Utilisateur ID : {self.identifiant}"

class Administrateur(Utilisateur):
    """Classe fille héritant d'Utilisateur avec redéfinition."""
    def __init__(self, identifiant, niveau_acces):
        super().__init__(identifiant) # Appel du constructeur parent
        self.niveau_acces = niveau_acces

    def obtenir_profil(self):
        # Redéfinition de la méthode héritée
        profil_base = super().obtenir_profil()
        return f"{profil_base} | Rôle : Admin (Niveau {self.niveau_acces})"

# Instanciation et test des objets
admin = Administrateur("USR-001", 3)
print(admin.obtenir_profil())
```

Exercices

1. **Exercice 1 :** Créez une classe `Livre` possédant un constructeur initialisant un titre et un auteur, ainsi qu'une méthode retournant une description textuelle de l'ouvrage.
2. **Exercice 2 :** Développez une classe fille `LivreNumerique` qui hérite de la classe `Livre` en y ajoutant un attribut supplémentaire pour la taille du fichier en mégaoctets, puis instanciez un objet de cette classe.

Chapitre 16 : Accès aux bases de données

L'accès aux bases de données permet de persister, d'interroger et de structurer des volumes importants d'informations de manière sécurisée en Python. Maîtriser ces concepts est indispensable pour connecter vos applications à des systèmes de stockage relationnels. Dans ce chapitre :

- Concepts de base des bases de données relationnelles
 - Connexion et paramétrage via la connexion et le curseur
 - Gestion de la structure des données avec le DDL (tables)
 - Gestion des transactions et validation avec le `commit`
 - Manipulation des données avec le DML (`SELECT`, `WHERE`)
-

Concepts de base

Les bases de données relationnelles permettent de stocker des données tabulaires structurées composées de lignes et de colonnes. Elles garantissent l'intégrité, la rapidité de recherche et la concurrence d'accès aux informations.

Approche conceptuelle de l'interaction avec une base de données
Utilisation généralement du module intégré sqlite3 en Python

Privilégiez l'utilisation des bases de données relationnelles dès que vos besoins de persistance impliquent des relations complexes entre plusieurs entités.

Connexion et paramétrage - connexion, curseur

La connexion établit le pont entre l'application Python et le fichier ou serveur de base de données, tandis que le curseur sert d'intermédiaire pour exécuter les requêtes SQL.

```
import sqlite3
```

```
# Établissement de la connexion et création du curseur  
connexion = sqlite3.connect("ma_base.db")  
curseur = connexion.cursor()
```

Pensez toujours à fermer explicitement votre curseur et votre connexion à la fin des traitements pour libérer les ressources système verrouillées.

Gestion de la Structure de données - Table DDL

Le langage de définition de données (DDL) permet de créer, modifier ou supprimer la structure des tables au sein de la base de données (instructions CREATE TABLE, etc.).

Création d'une table relationnelle via une requête DDL

```
curseur.execute("""
    CREATE TABLE IF NOT EXISTS utilisateurs (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        nom TEXT NOT NULL,
        age INTEGER
    )
""")
```

Définissez rigoureusement les types de données et les contraintes (NOT NULL, PRIMARY KEY) dès la conception de vos tables pour garantir l'intégrité des informations.

Gestion des transactions - commit

La gestion des transactions permet de valider définitivement un ensemble d'opérations en base de données grâce à l'instruction commit, garantissant la cohérence globale des données.

Validation des modifications apportées à la base de données

```
connexion.commit()
```

En cas d'erreur lors d'une transaction, utilisez l'instruction rollback pour annuler les modifications en cours et rétablir l'état stable précédent de la base.

Manipulation des données - Select/Where DML

Le langage de manipulation des données (DML) permet d'insérer, de modifier, de supprimer et de rechercher des enregistrements à l'aide des instructions SELECT et de la clause WHERE.

Requête DML pour sélectionner des enregistrements filtrés

```
curseur.execute("SELECT nom, age FROM utilisateurs WHERE age >= ?",
(18,))
resultats = curseur.fetchall()
```

Utilisez toujours des requêtes paramétrées (avec des points d'interrogation ?) pour injecter des variables afin de vous prémunir totalement contre les failles d'injection SQL.

Exemple de synthèse

```
import sqlite3

def gerer_base_de_donnees():
    """Programme complet combinant connexion, DDL, transactions et
    requêtes DML."""
    # 1. Connexion et paramétrage
    connexion = sqlite3.connect("entreprise.db")
    curseur = connexion.cursor()

    # 2. Gestion de la structure de données (DDL)
    curseur.execute("""
        CREATE TABLE IF NOT EXISTS employes (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            nom TEXT,
            salaire REAL
        )
    """)

    # 3. Insertion de données (DML) et gestion des transactions
    (commit)
    curseur.execute("INSERT INTO employes (nom, salaire) VALUES
    (?, ?)", ("Alice", 2500.0))
    curseur.execute("INSERT INTO employes (nom, salaire) VALUES
    (?, ?)", ("Bob", 3100.0))
    connexion.commit() # Validation de la transaction

    # 4. Manipulation des données avec SELECT et WHERE (DML)
    curseur.execute("SELECT nom, salaire FROM employes WHERE salaire >
    ?", (2800.0,))
    recrutements_hauts = curseur.fetchall()

    for employe in recrutements_hauts:
        print(f"Employé qualifié : {employe[0]} avec un salaire de
        {employe[1]}€")

    # Fermeture propre des ressources
    curseur.close()
    connexion.close()

# Exécution de la fonction de synthèse
gerer_base_de_donnees()
```

Exercices

1. **Exercice 1 :** Écrivez un script Python qui utilise le module `sqlite3` pour créer une base de données, instancier une table `produits` (contenant un ID, un nom et un prix), puis y insérer un enregistrement validé par un `commit`.
2. **Exercice 2 :** Rédigez une requête `SELECT` associée à une clause `WHERE` pour récupérer et afficher tous les produits dont le prix est inférieur à un certain seuil depuis la table créée à l'exercice précédent.